Assignment — Problem Statement Responses

Sai Sri Prakash Chennoju |
prakashsaisri9@gmail.com | +91-7842914419

Question 1 — Self-Rating on LLM, Deep Learning, AI & ML

This question asks me to honestly reflect on where I stand across four interconnected but distinct areas of AI. Coming out of a four-year CSE programme with an AI/ML specialisation, I have had exposure to all four — but the depth is not uniform. The rating scale is: **A** = can code independently, **B** = can code under supervision, **C** = little or no understanding. My self-assessment below is based on what I have actually built, not just what I have studied in class.

Domain	Rating	What I can do	Experience / Evidence
Machine Learning	B	Can implement models and pipelines under guidance	AutoML tool (EDA + regression/classification), TB disease classification using RF & XGBoost, feature selection via permutation importance
Deep Learning	B	Can build and train models with supervision	Hand-sign detection (OpenCV, keypoint/edge detection), PyTorch usage during SRM AP research internship
AI	B	Can apply AI concepts to real problems with guidance	Telugu LLM development at Viswam.AI, image analysis and medical AI research at SRM University AP
LLM	B / C	Conceptual understanding + early hands-on experience; actively improving	Viswam.AI Summer of AI 2025: tokenization, training pipelines, and evaluation for a regional language LLM

Machine Learning — B

ML is where I am most comfortable. Over the course of my degree I moved beyond just running sklearn tutorials and actually built a full AutoML tool that handles exploratory data analysis, model selection, training, and evaluation for both regression and classification tasks. More importantly, during my research internship at SRM University AP, I applied ML to a real medical problem — classifying Tuberculosis patients from healthy controls using gene expression data. I used RandomForestClassifier and XGBoost, compared their performance through accuracy, precision, recall, and confusion matrix analysis, and identified significant genes using permutation importance. That project taught me the full lifecycle of an ML problem: data preprocessing, feature engineering, model training, evaluation, and interpretation of results. I can comfortably work through all of these stages with some guidance on edge cases.

Deep Learning — B

My Deep Learning experience is primarily through computer vision. During the same research internship, I built a hand-sign detection system using OpenCV that used keypoint detection and edge detection techniques to recognize gestures without physical contact — the goal was touchless human-computer interaction. I used PyTorch for parts of the feature extraction and model training pipeline. The project gave me solid exposure to how raw image data is processed, how features are extracted, and how models are evaluated in a real deployment context. I am not yet at a point where I would build a complex architecture like a Transformer from scratch independently, but I can work through CV pipelines and fine-tuning tasks with guidance.

Artificial Intelligence — B

My AI confidence comes from two directions. The first is the hands-on research work described above, which involved applying AI to image analysis and medical classification. The second — and more impactful — was

my internship at Viswam.AI, where I contributed to the Summer of AI 2025 programme and worked on what is described as the world's first foundational Telugu Large Language Model. Being part of that project gave me a much broader picture of how large-scale AI systems are architected and trained, particularly for low-resource regional languages where standard English-first assumptions break down.

LLM — B / C

LLMs are the area I am most actively learning. Before Viswam.AI, my understanding was largely theoretical — I knew what transformers were, had read about attention mechanisms, but had not built anything end-to-end myself. After the internship, where I worked on tokenization strategies, training pipelines, and evaluation for Telugu, I would place myself at an honest B/C — closer to B in terms of understanding and conceptual ability, but still needing guidance to build a production-quality LLM system independently. I see this as my highest-growth area going forward.

Question 2 — Key Architectural Components of an LLM-Based Chatbot

At first glance, building a chatbot with an LLM sounds simple — you send a message to an API and get a response back. But once you try to build one that is actually useful, reliable, and context-aware, you quickly realise there is a whole pipeline of components working together. This question asks me to explain that pipeline at a high level, which I will do drawing on both what I have studied and what I learned during my work at Viswam.AI.

```
[ User sends a message via UI ]
↓
[ Embedding model converts query to a vector ]
↓
[ Vector DB retrieves top-K relevant document chunks ]
↓
[ Prompt Builder: system prompt + context + history + user query ]
↓
[ LLM generates natural language response ]
↓
[ Memory updated → response returned to user ]
```

End-to-end pipeline of an LLM-based chatbot with RAG

Component	What It Does	Tools / Examples
LLM (Core Model)	Reads the fully assembled prompt and generates the natural language response	GPT-4, Claude 3, LLaMA 3, Gemini
Embedding Model	Converts text into dense numerical vectors for similarity search	all-MiniLM-L6-v2, text-embedding-ada-002
Vector Database	Stores all document embeddings; retrieves most relevant chunks at query time	FAISS, Chroma, Pinecone, Weaviate
Prompt Builder	Assembles system prompt + retrieved context + memory + current user query	LangChain, LlamaIndex
Memory Module	Maintains conversation history so the bot can reference earlier turns	ConversationBufferMemory, Redis
API / UI Layer	Exposes the chatbot to end users via web, mobile, or desktop interface	FastAPI, Streamlit, React, Flutter

Core components of an LLM chatbot and their roles

The LLM — Core Engine

The LLM is the brain of the system. It takes a natural language prompt as input and produces a natural language response. Models like GPT-4, Claude, or LLaMA have been pre-trained on enormous datasets and can understand context, follow instructions, and generate coherent text. However, one critical limitation is that an LLM's knowledge is frozen at its training cutoff — it knows nothing about your specific documents, your organisation's data, or events after that date. This is why we do not just send user queries directly to the LLM; we first enrich the prompt with relevant information retrieved from our own data sources.

RAG — Retrieval-Augmented Generation

RAG is the approach that solves the LLM's knowledge limitation. The idea is to maintain your own searchable knowledge base of documents, and before every LLM call, search that knowledge base for content relevant to the user's query. The retrieved content is then injected into the prompt so the LLM can use it to generate a grounded, accurate response. This dramatically reduces hallucination — the tendency of LLMs to confidently make up facts — and makes the chatbot genuinely useful for domain-specific applications like customer support, medical Q&A, or internal knowledge management.

RAG works in two phases. During **indexing**, all source documents are split into chunks, each chunk is converted to a vector embedding using an embedding model, and those vectors are stored in a vector database. During **retrieval**, the user's query is embedded using the same model, and the vector database returns the most semantically similar chunks. These chunks become the context that the LLM reasons over when generating its response.

The Prompt Builder — Where Quality Happens

The prompt builder is often underestimated, but it is where the practical quality of the chatbot is determined. A well-designed prompt assembles four things in the right order: (1) a **system prompt** that tells the LLM its role, tone, and constraints; (2) the **retrieved context** from the vector database; (3) the **conversation history** from the memory module; and (4) the **current user query**. Getting this structure right — while staying within the model's token limit — is a real engineering challenge. Too little context leads to vague answers; too much causes the model to lose focus.

Memory — Making It a Real Conversation

An LLM has no memory between calls — every request is stateless by design. Without a memory module, the chatbot would forget everything the user said the moment a new message arrives. The simplest solution is to pass the last N conversation turns as part of every prompt. For longer sessions, a summarisation strategy is used: older turns are compressed into a short summary so the token budget is not exhausted. More advanced systems use external stores like Redis to persist memory across sessions and devices.

A Note from My Experience — Language Matters

At Viswam.AI, I learned something that does not appear in most chatbot architecture diagrams: **the choice of embedding model is language-critical**. Standard English embedding models like ada-002 perform poorly on Telugu text because they were not trained on it. For a multilingual or regional-language chatbot, you need embedding models specifically trained on or fine-tuned for that language. This is a decision that must be made at the architecture stage, not as an afterthought.

Question 3 — Vector Databases: Explanation & Choice

What is a Vector Database?

To understand why vector databases exist, it helps to first understand what they are solving. Traditional relational databases are excellent at exact lookups — find me all records where the city is 'Hyderabad' or the age is greater than 25. But they completely fail at semantic search: *find me documents that are about the same topic as this query*, even if they use different words. There is no SQL clause for that.

The solution comes from how modern ML models represent meaning. When you pass a sentence through an embedding model — a neural network trained on large text corpora — it outputs a dense list of floating-point numbers, typically 384 or 768 values long. This list is called a **vector embedding**. The key property is that sentences with similar meanings produce vectors that are close to each other in that high-dimensional space. For example, the sentences *"fever and body ache"* and *"common flu symptoms"* would produce very similar vectors, even though they share no words. A traditional database cannot capture this; a vector database is built specifically for it.

A vector database stores these embeddings alongside the original content, and when you give it a query, it converts that query into a vector using the same embedding model and returns the stored items whose vectors are closest — this is called **Approximate Nearest Neighbor (ANN) search**. The result is a search engine that understands meaning, not just keywords.

```
Query:  "explain overfitting in machine learning"
                ↓  Embedding Model (all-MiniLM-L6-v2)  ↓
Vector: [0.21, -0.44, 0.87, 0.03, -0.12, ...]   (384 dimensions)
                ↓  ANN Search (Cosine Similarity) in FAISS  ↓
    Result 1: 'regularization prevents model overfitting...'
  Result 2: 'bias-variance tradeoff in supervised learning...'
      Result 3: 'dropout as a regularization technique...'
```

How a vector database processes a semantic query — same meaning, different words

How Similarity is Measured

There are three common distance metrics used in vector databases:

- **Cosine Similarity —** Measures the angle between two vectors. The most popular metric for text because it is not affected by the length of the document — only by its direction in vector space. A score of 1.0 means identical meaning; 0.0 means completely unrelated.
- **Euclidean Distance (L2) —** Measures the straight-line geometric distance between two vector points. Commonly used in image embeddings and spatial data.
- **Dot Product —** A faster computation often used in dense retrieval systems. Sensitive to vector magnitude, so it tends to favour longer, more informative documents.

Why This Matters for LLM Applications

Vector databases have become a fundamental building block in modern AI systems precisely because of RAG (Retrieval-Augmented Generation). When you want a chatbot to answer questions based on your own documents, you cannot just feed all the documents into the prompt — most LLMs have context limits and large-scale injection is slow and expensive. Instead, you index all documents into a vector database, and at query time, retrieve only the two or three most relevant chunks. This keeps prompts lean, responses accurate, and the system scalable.

Popular Vector Databases — Comparison

Database	Free?	Setup	Speed	Best For	Verdict
FAISS	Yes	pip install	Very Fast	Local projects, research, RAG prototypes	★ My Choice
Chroma	Yes	pip install	Fast	LLM apps, LangChain RAG	Close 2nd
Pinecone	No (SaaS)	Cloud + API key	Fast	Production apps at scale	Paid / cloud only
Weaviate	Yes	Docker / hosted	Fast	Hybrid keyword + vector search	Heavier setup
Qdrant	Yes	Docker / cloud	Very Fast	Filtered vector search at scale	Strong alternative

Comparison of commonly used vector databases

Hypothetical Problem

> **Problem: Smart Lecture Notes Search Assistant**
>
> A final-year engineering student has accumulated 4 years of lecture PDFs, scanned handwritten notes, and solved assignment PDFs — roughly 4,000 to 5,000 text chunks in total. They want to ask natural language questions like "what did we cover about CNNs in deep learning?" or "explain the difference between precision and recall from our ML notes" and instantly receive the 3–5 most relevant chunks from their own notes. These chunks are then passed to an LLM (like GPT or Claude) which generates a clean, contextual answer. The system runs locally on a laptop — no cloud infrastructure, no paid accounts.

My Choice: FAISS

For this problem, I would choose **FAISS — Facebook AI Similarity Search**, open-sourced by Meta AI Research. FAISS is the most widely used and well-documented vector search library available, used in production systems at companies like Meta, Spotify, and Airbnb, and extensively cited in academic ML research. For a student-scale project like this, it is the perfect fit.

The most practical reason is the **zero-friction setup**. There is no server to run, no Docker container, no cloud account, and no API key. A single pip install is all that is needed. Every part of the pipeline — embedding, indexing, and searching — runs entirely on a local machine. For a personal knowledge base of 5,000 chunks, this is more than sufficient.

Beyond convenience, FAISS also offers excellent **depth for learning**. It exposes different index types — IndexFlatL2 for exact search, IndexIVFFlat for fast approximate search on larger datasets, and HNSW for graph-based search — which helped me understand the actual trade-offs between search accuracy and speed. This understanding is transferable to any other vector database. FAISS is also the library that most research papers use when benchmarking vector search, so learning it well also helps with reading the literature.

```
# Install
pip install faiss-cpu sentence-transformers

from sentence_transformers import SentenceTransformer
import faiss, numpy as np

# Step 1: Encode all document chunks
model   = SentenceTransformer('all-MiniLM-L6-v2')
chunks  = ['CNN lecture notes...', 'Overfitting means...', ...]  # your docs
```

```
embeds  = model.encode(chunks).astype('float32')    # shape: (N, 384)

# Step 2: Build and save the FAISS index
index   = faiss.IndexFlatL2(384)                    # exact L2 search
index.add(embeds)
faiss.write_index(index, 'notes.index')             # persist to disk

# Step 3: Search at query time
query   = model.encode(['What is overfitting?']).astype('float32')
D, I    = index.search(query, k=5)                  # top-5 results
results = [chunks[i] for i in I[0]]                 # retrieve text
```

Complete FAISS pipeline — embed, index, and search in under 20 lines of Python

Why Not the Others?

- **Pinecone —** Excellent managed service, but it requires a paid subscription and an internet connection. Running a personal study tool on a cloud database introduces unnecessary latency and cost when everything can run locally.
- **Chroma —** Honestly a very close second. It is arguably even simpler to plug into a LangChain RAG pipeline, and it is also free and open source. The reason I lean toward FAISS here is that it gave me a lower-level understanding of how indexing works, which I found valuable. If the project were primarily a LangChain demo, I might choose Chroma instead.
- **Weaviate / Qdrant —** Both are powerful and open source, but they require running a separate server process. For a laptop-based personal project, that overhead is unnecessary. They become the right choice when the application needs horizontal scaling, metadata filtering at scale, or multi-tenant access.

Scaling Up

If this project were to grow — for example, shared across a university department with hundreds of users and real-time document uploads — the migration path is clear. FAISS does not support real-time inserts well and has no built-in server mode. In that scenario, I would move to **Qdrant** for a self-hosted, open-source option with rich filtering and streaming updates, or to **Pinecone** for a fully managed cloud option. Thanks to LangChain's unified vector store interface, this migration would require changing only a few lines of code.